Intro to Reinforcement Learning Theory

CS245: Online Optimization and Learning

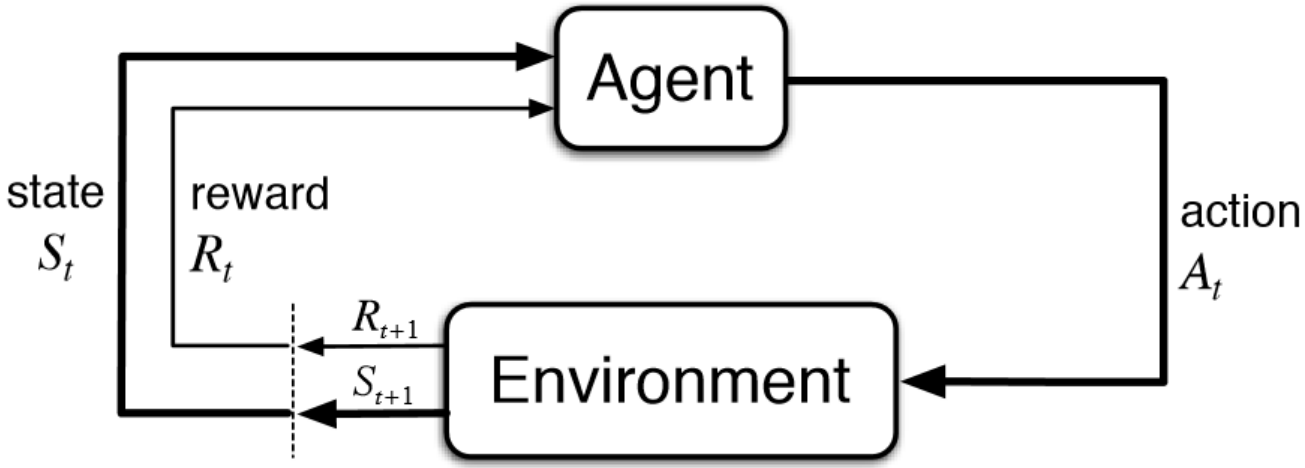
Xin Liu
SIST, ShanghaiTech University

Practical Applications of Reinforcement Learning

Reinforcement learning:

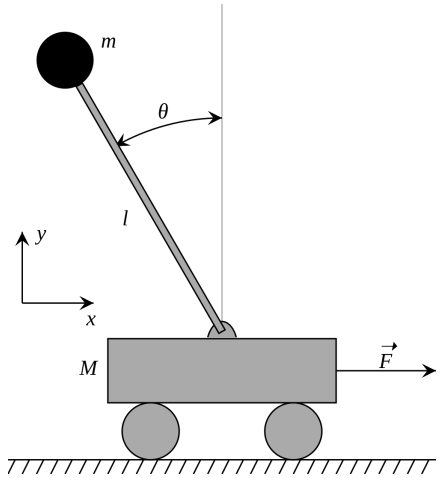
- Data Center Cooling
- AlphaStar in StarCraft
- AlphaGo
- DeepSeek

Reinforcement Learning (RL): Cart Pole

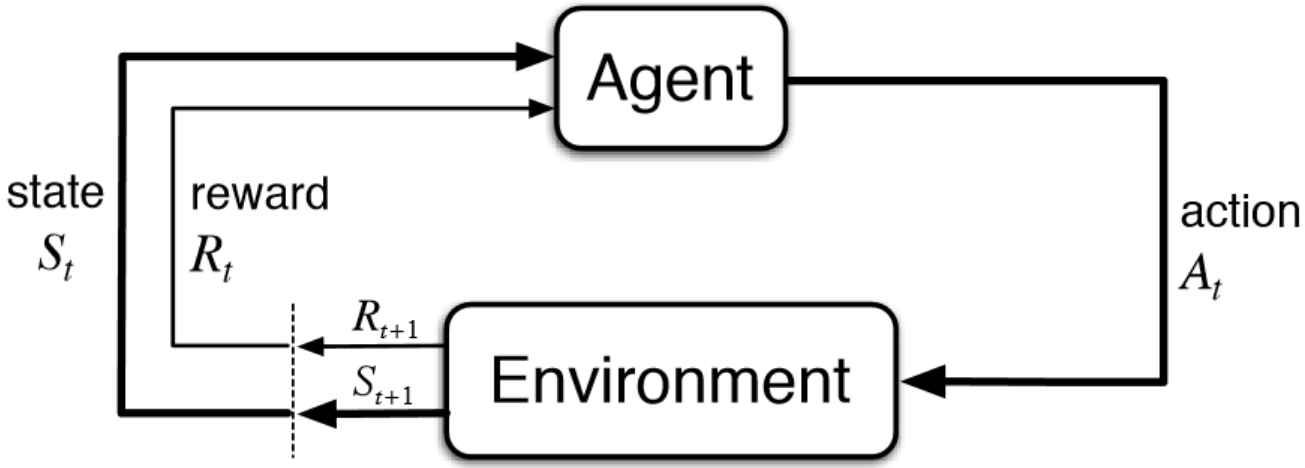


Objective: Balance a pole with a movable cart.

- State: angle, position, horizontal velocity, ...
- Action: horizontal force
- Reward: unit reward at each time step when the pole is upright

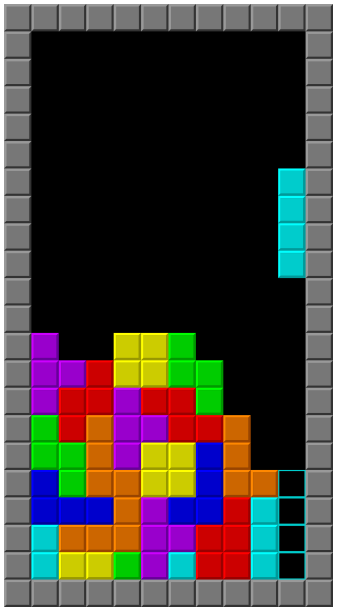


Reinforcement Learning (RL): Tetris

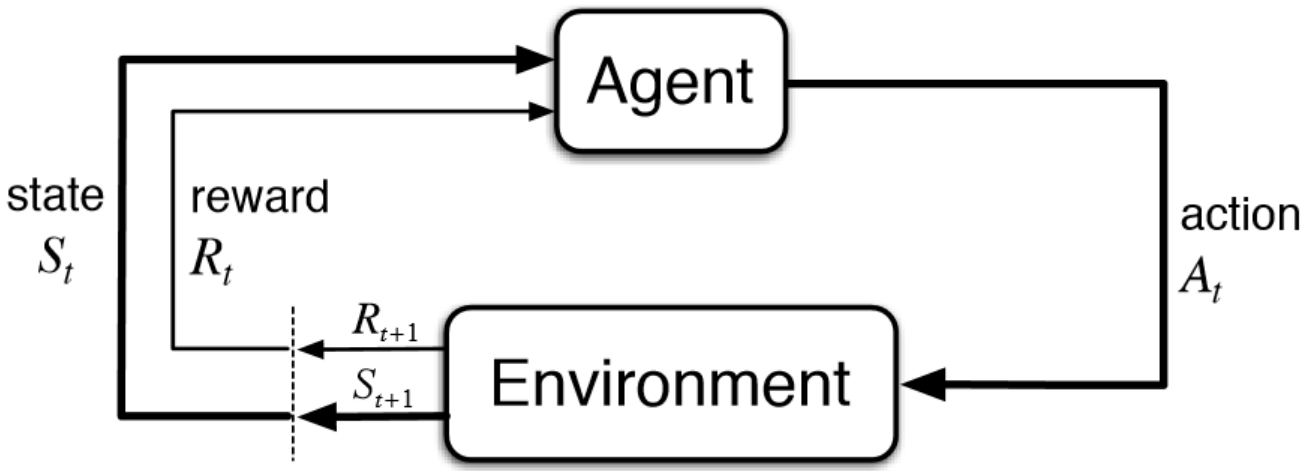


Objective: stay in the game as long as possible.

- State: the state of the tiles
- Action: orientation of the falling piece
- Reward: unit reward at each time staying in the game

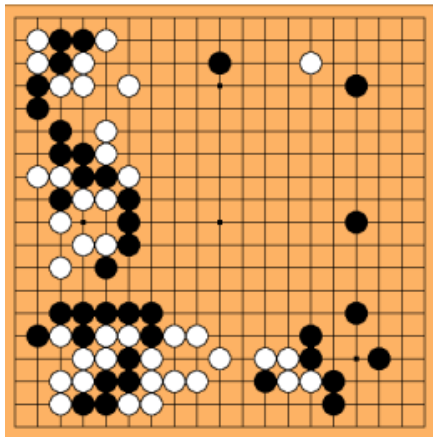


Reinforcement Learning (RL): Go

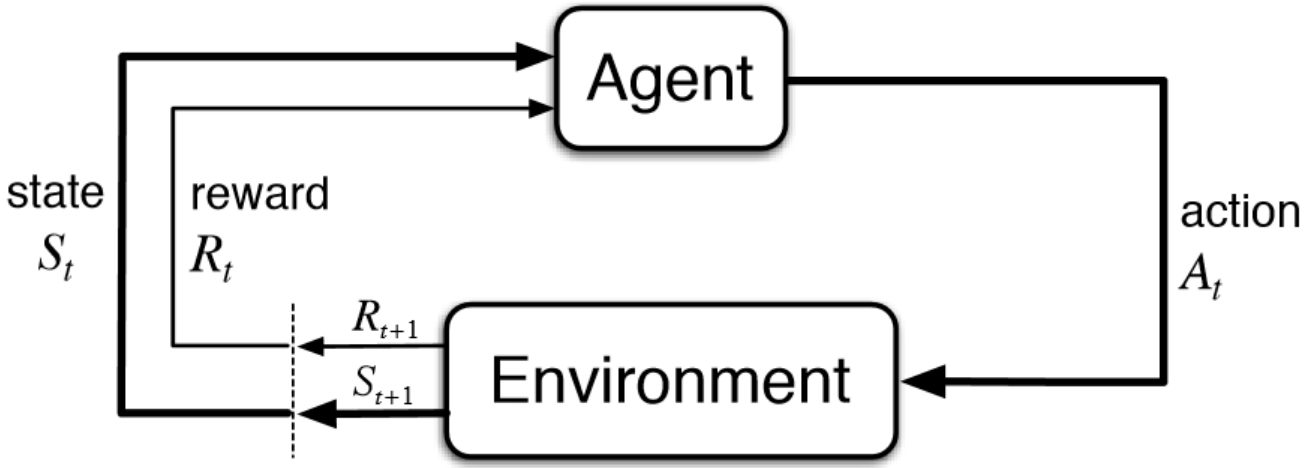


Objective: win the game.

- State: position of all pieces
- Action: location of the next piece
- Reward: 1 when winning the game and 0 otherwise



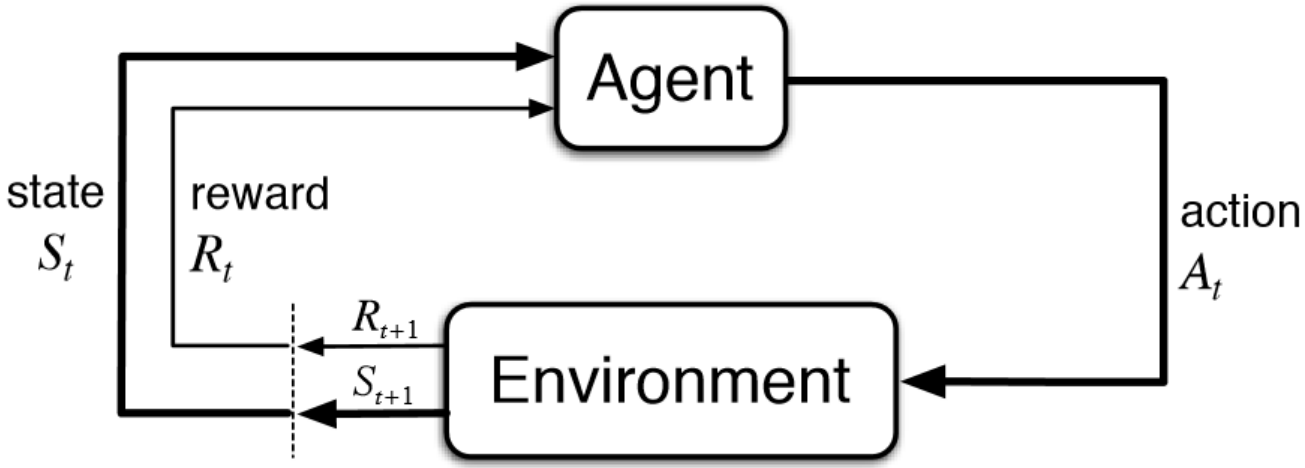
Reinforcement Learning (RL): Self-Driving



Objective: safe and efficient transportation.

- State: position of the car and the surrounding environment
- Action: steering, brakes, accelerates
- Reward: reward when keeping in the lane, avoid accidents

Reinforcement Learning (RL): Important Features



- Discover optimal actions by trying them (trial-and-error search)
- Current action affects next state
- Maximize long-term return instead of instantaneous rewards

What is RL?

RL is a principle method to solve (approximately)

(Optimal) Bellman Equation!

What is RL?

RL is a principle method to solve (approximately)

(Optimal) Bellman Equation!

Value-based policies:

- Find the optimal policy by finding optimal value functions.
- TD-learning, Q-learning, Deep Q-learning.

What is RL?

RL is a principle method to solve (approximately)

(Optimal) Bellman Equation!

Value-based policies:

- Find the optimal policy by finding optimal value functions.
- TD-learning, Q-learning, Deep Q-learning.

Policy-based policies

- Find the optimal policy by directly optimizing the policy.
- Policy gradient, TRPO, PPO.

What is RL?

RL is a principle method to solve (approximately)

(Optimal) Bellman Equation!

Value-based policies:

- Find the optimal policy by finding optimal value functions.
- TD-learning, Q-learning, Deep Q-learning.

Policy-based policies

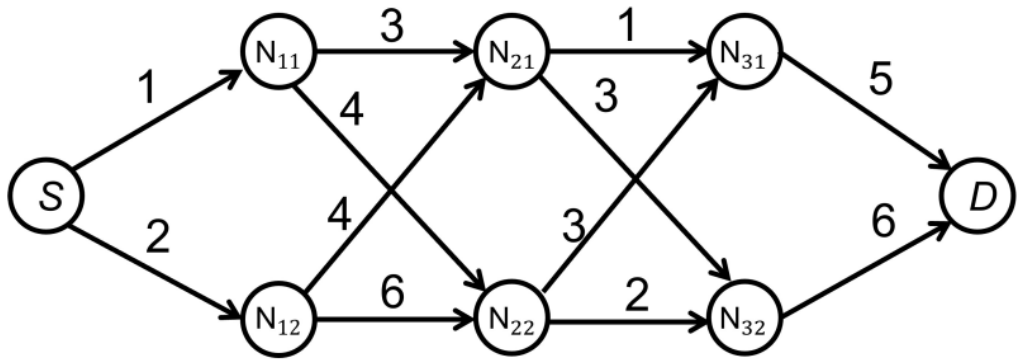
- Find the optimal policy by directly optimizing the policy.
- Policy gradient, TRPO, PPO.

A few performance guarantee and two important works:

- Is Q-learning Efficent?
- On the Theory of Policy Gradient Methods: Optimality, Approximation, and Distribution Shift

Dynamic Programming and Bellman Equation

Find the most rewarding path, i.e. the path with the maximum reward, from S to D.



Exclusive search (forward search): number of possible paths?

DP (backward search): start from the destination.

Dynamic Programming and Bellman Equation

Principle of Optimality: A subsolution of an optimal solution is the optimal solution for the corresponding subproblem.

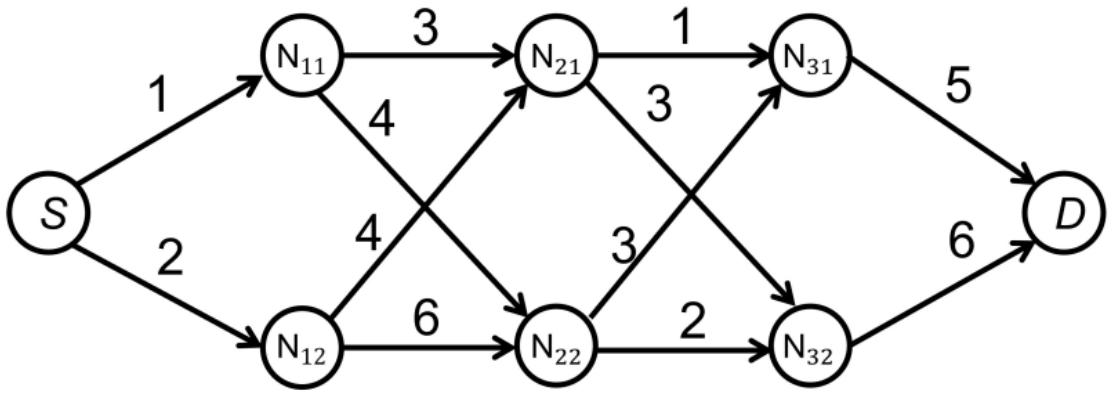
Bellman Equation:

$$V_k(j) = \max_{i \in \mathcal{N}(j)} R_k(j,i) + V_{k+1}(i)$$

$\mathcal{N}(j)$ is set of node j's neighbors at stage $k+1$, $R_k(j,i)$ is the reward of edge (j,i) and $V_k(i)$ is the maximum reward to reach destination D from node j.

Stochastic Dynamic Programming and Bellman Equation

Find the most rewarding path, i.e. the path with the maximum reward, from S to D.



Stochastic transition:

- For action up: $\mathbb{P}(s_{k+1} = up) = 0.9$ & $\mathbb{P}(s_{k+1} = down) = 0.1$.
- For action down: $\mathbb{P}(s_{k+1} = up) = 0.1$ & $\mathbb{P}(s_{k+1} = down) = 0.9$.

Optimal policy: State-dependent policy $\pi(a_k|s_k)$.

Stochastic Dynamic Programming and Bellman Equation

Value function for a policy:

$$V^{\pi}(s_1) = \mathbb{E}[R_T(s_T) + \sum_{k=1}^{T-1} R(s_k, a_k)],$$

$$V^{\pi}(s_1) = \mathbb{E}[R(s_1, a_1) + \gamma R(s_2, a_2) + \gamma^2 R(s_3, a_3) + \cdots].$$

Our goal is to design a policy $\pi(a_k|s_k)$ such that the action sequence

$$\{a_1, a_2, \cdots, a_{T-1}\} \quad \text{or} \quad \{a_1, a_2, \cdots\}$$

to maximize

$$\max_{\pi} V^{\pi}(s_1).$$

Stochastic Dynamic Programming and Bellman Equation

Bellman Equation:

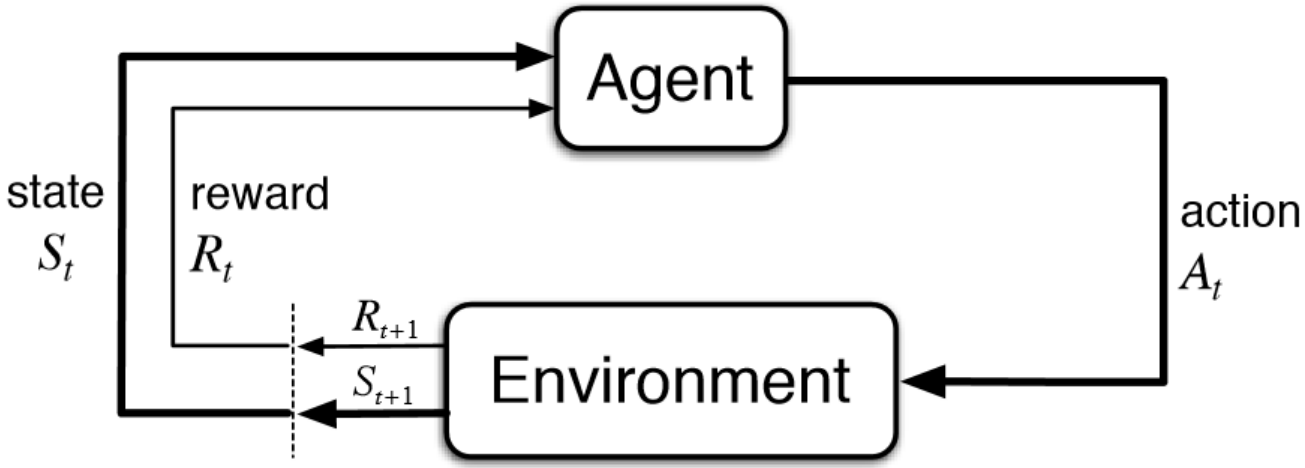
$$V_k^*(s_k) = \max_{a_k} \mathbb{E}[R_k(s_k, a_k) + V_{k+1}^*(s_{k+1})],$$
$$V^*(s) = \max_a \mathbb{E}[R(s, a) + \gamma V^*(s')].$$

Given the optimal value function, we can also recover the optimal policy:

$$a_k = \arg\max_{a_k} \mathbb{E}[R_k(s_k, a_k) + V_{k+1}^*(s_{k+1})],$$
$$a = \arg\max_a \mathbb{E}[R(s, a) + \gamma V^*(s')].$$

Note we need to take the expectation w.r.t. the stochastic environment and policy.

RL: Markov decision process (MDP)



Basic elements in MDP:

- State space $\mathcal{S}$
- Action space $\mathcal{A}$
- Immediate Reward $R(s, a)$
- Transition probabilities: $\mathbb{P}(\cdot|s, a)$
- Policy: $\pi(\cdot|s)$

Two types of RL:

- Episodic RL
- Infinite Discounted RL

Model-Based RL: Value Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Suppose that both the state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite, the model $\mathbb{P}(s'|s,a)$ and $r(s,a) = \mathbb{E}[R(s,a)]$ are known.

Model-Based RL: Value Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Suppose that both the state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite, the model $\mathbb{P}(s'|s,a)$ and $r(s,a) = \mathbb{E}[R(s,a)]$ are known.

Starting from $V_0(s)$, we can iteratively compute:

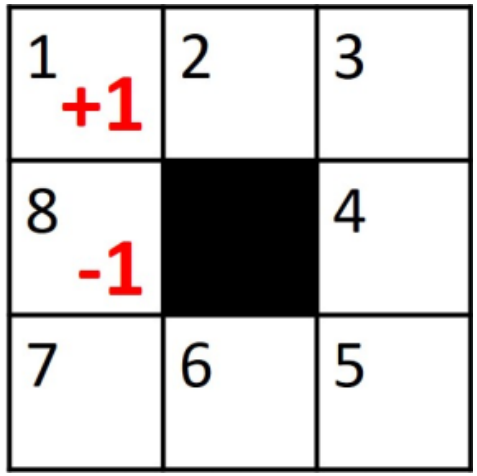
$$V_{k+1}(s) = \max_a r(s,a) + \gamma \sum_{s'} \mathbb{P}(s'|s,a) V_k(s').$$

Value Iteration: Grid World

Grid world example:

- Two actions: clockwise (c) or counter-clockwise (cc).
- The agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2.

Start with $V_0 = [0, 0, 0, 0, 0, 0, 0, 0]$.

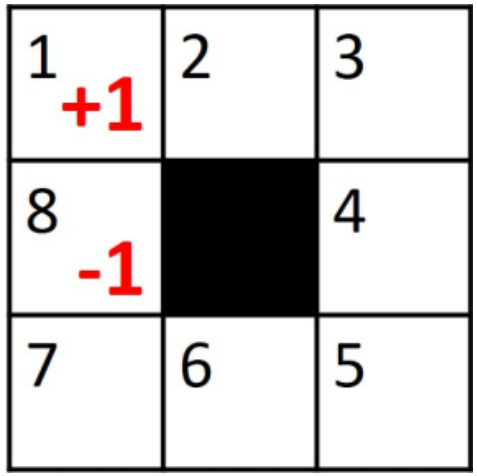


Value Iteration: Grid World

Grid world example:

- Two actions: clockwise (c) or counter-clockwise (cc).
- The agent follows the action with probability 0.8 and moves to the opposite direction with probability 0.2.

Start with $V_0 = [0, 0, 0, 0, 0, 0, 0, 0]$.



End with $V_\infty = [3.36, 2.86, 2.43, 2.07, 1.77, 1.54, 1.49, 1.69]$.

Model-Based RL: Value Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Suppose that both the state space $\mathcal{S}$ and action space $\mathcal{A}$ are finite, the model $\mathbb{P}(s'|s,a)$ and $r(s,a) = \mathbb{E}[R(s,a)]$ are known.

Starting from $V_0(s)$, we can iteratively compute:

$$V_{k+1}(s) = \max_a r(s,a) + \gamma \sum_{s'} \mathbb{P}(s'|s,a) V_k(s').$$

Can value iteration converge to the optimal value functions?

$$V_k(s) \to V^*(s)?$$

Model-Based RL: Policy Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Model-Based RL: Policy Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Policy evaluation (prediction) for π_k:

$$V^{\pi_k}(s) = \mathbb{E}[R(s,a)] + \gamma \sum_a \pi_k(a|s) \sum_{s'} \mathbb{P}(s'|s,a) V^{\pi_k}(s').$$

Model-Based RL: Policy Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Policy evaluation (prediction) for π_k:

$$V^{\pi_k}(s) = \mathbb{E}[R(s,a)] + \gamma \sum_a \pi_k(a|s) \sum_{s'} \mathbb{P}(s'|s,a) V^{\pi_k}(s').$$

Policy improvement for π_{k+1}:

$$\pi_{k+1} = \arg\max_\pi \mathbb{E}[R(s,a)] + \gamma \sum_a \pi(a|s) \sum_{s'} \mathbb{P}(s'|s,a) V^{\pi_k}(s').$$

Model-Based RL: Policy Iteration

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')].$$

Policy evaluation (prediction) for π_k:

$$V^{\pi_k}(s) = \mathbb{E}[R(s,a)] + \gamma \sum_a \pi_k(a|s) \sum_{s'} \mathbb{P}(s'|s,a) V^{\pi_k}(s').$$

Policy improvement for π_{k+1}:

$$\pi_{k+1} = \arg\max_\pi \mathbb{E}[R(s,a)] + \gamma \sum_a \pi(a|s) \sum_{s'} \mathbb{P}(s'|s,a) V^{\pi_k}(s').$$

Can value iteration converge to the optimal value functions?

$$\pi_k \to \pi^*?$$

Model-Based RL: Challenges

Model-based RL methods are good and optimal! Just use either value iteration or policy iteration?

Let's look at the assumptions for model-based methods:

- Exact knowledge of reward and $\mathbb{P}(s'|s,a)$.
- Finite state and action spaces.

AlphaGo satisfies these assumptions?

Model-Free RL

Bellman Equation for infinite discounted MDP:

$$V^*(s) = \max_a \mathbb{E}[R(s,a) + \gamma V^*(s')],$$
$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^*(s').$$

Suppose we have $Q^*(s,a)$, we just take the (optimal) action according to

$$a^* \in \arg\max_a Q^*(s,a).$$

The key question is how to learn the optimal $Q^*(s,a)$ in a model-free style?

Model-free RL: SARSA

Again recall the Optimal Bellman equation:

$$\begin{aligned} Q^{\pi^*}(s,a) =& r(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^{\pi^*}(s') \\ =& r(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) \max_{a'} Q^{\pi^*}(s',a') \end{aligned}$$

We have samples like $(s, a, R(s,a), s', a')$ and do approximation

$$Q^{\pi}(s,a) \approx R(s,a) + \gamma Q^{\pi}(s',a'),$$

which gives us the well-known SARSA algorithm

$$Q^{\pi}(s,a) = Q^{\pi}(s,a) + \alpha \left(R(s,a) + \gamma Q^{\pi}(s',a') - Q^{\pi}(s,a) \right).$$

Model-free RL: SARSA

SARSA (1994)

Initialize Q table and a state s_0.
At every step k, we choose the action with $\epsilon-$greedy with $1-\epsilon$ prob

$$a_k \in \arg\max_a Q(s_k, a).$$

Observe $R(s_k, a_k)$ with the data $(s_{k-1}, a_{k-1}, R(s_k, a_k), s_k, a_k)$ and update the Q as follows:

$$Q(s_{k-1}, a_{k-1}) = (1-\alpha)Q(s_{k-1}, a_{k-1}) + \alpha\left[R(s_k, a_k) + \gamma Q(s_k, a_k)\right].$$

Until convergencee or early stop.

Model-free RL: Q-learning

Again recall the Optimal Bellman equation:

$$\begin{aligned} Q^{\pi^*}(s,a) =& r(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) V^{\pi^*}(s') \\ =& r(s,a) + \gamma \sum_{s'} \mathcal{P}(s'|s,a) \max_{a'} Q^{\pi^*}(s',a') \end{aligned}$$

We have samples like $(s, a, R(s,a), s')$ and do approximation

$$Q^{\pi}(s,a) \approx R(s,a) + \gamma \max_{a'} Q^{\pi}(s',a'),$$

which gives us the well-known Q-learning algorithm

$$Q^{\pi}(s,a) = Q^{\pi}(s,a) + \alpha \left(R(s,a) + \gamma \max_{a'} Q^{\pi}(s',a') - Q^{\pi}(s,a) \right).$$

Model-free RL: Q-learning

Q-learning (1989)

Initialize Q table and an state s_0.

At every step k, we choose the action with $\epsilon-$greedy with $1-\epsilon$ prob

$$a_k \in \arg\max_a Q(s_k, a).$$

Observe $R(s_k, a_k)$ and s_{k+1}. Update the Q as follows:

$$Q(s_k, a_k) = (1-\alpha)Q(s_k, a_k) + \alpha\left[R(s_k, a_k) + \gamma \max_a Q(s_{k+1}, a)\right].$$

Until convergence or early stop.

Deep Q learning

"Q-learning + neural networks + many tricks":

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Model-free RL: Q-learning

Q-learning pros:

- Off-policy, utilize the historical data, and sample efficient
- Computation efficient

Q-learning cons:

- Too "optimistic", over-estimation, instablity
- Discrete and finite action spaces

Q-learning in MDP

Q-learning with UCB algorithm[1]:

initialize $Q_h(x,a) \leftarrow H$ and $N_h(x,a) \leftarrow 0$ for all $(x,a,h) \in \mathcal{S} \times \mathcal{A} \times [H]$.
for episode $k = 1, \ldots, K$ **do**
  receive x_1.
  for step $h = 1, \ldots, H$ **do**
    Take action $a_h \leftarrow \operatorname{argmax}_{a'} Q_h(x_h, a')$, and observe x_{h+1}.
    $t = N_h(x_h, a_h) \leftarrow N_h(x_h, a_h) + 1$; $b_t \leftarrow c\sqrt{H^3\iota/t}$.
    $Q_h(x_h, a_h) \leftarrow (1-\alpha_t)Q_h(x_h, a_h) + \alpha_t[r_h(x_h, a_h) + V_{h+1}(x_{h+1}) + b_t]$.
    $V_h(x_h) \leftarrow \min\{H, \max_{a' \in \mathcal{A}} Q_h(x_h, a')\}$.

Optimistic Q-learning:

- Maintain each counter for each state-action pair (arm).
- Add UCB bonus in the "future reward" (exploration).
- Choose greedy action (exploitation).

[1] Jin, C., et al. "Is Q-learning provably efficient?", NIPS 2018.

Theoretical Results of Q-learning

Theorem 1 (Watkins and Dayan 1992)

Q-learning converges to the optimum action-values with probability 1 *so long as all actions are repeatedly sampled in all states.*

Theorem 2 (Jin et. al 2018)

Under Q-learning with UCB, by carefully choosing learning rates, the regret is bound as

$$\sum_{t=k}^{K} V_1^*(s) - V_1^{\pi_k}(s) = O\left(\sqrt{H^4|\mathcal{S}||\mathcal{A}|K\log(|\mathcal{S}||\mathcal{A}|)}\right).$$

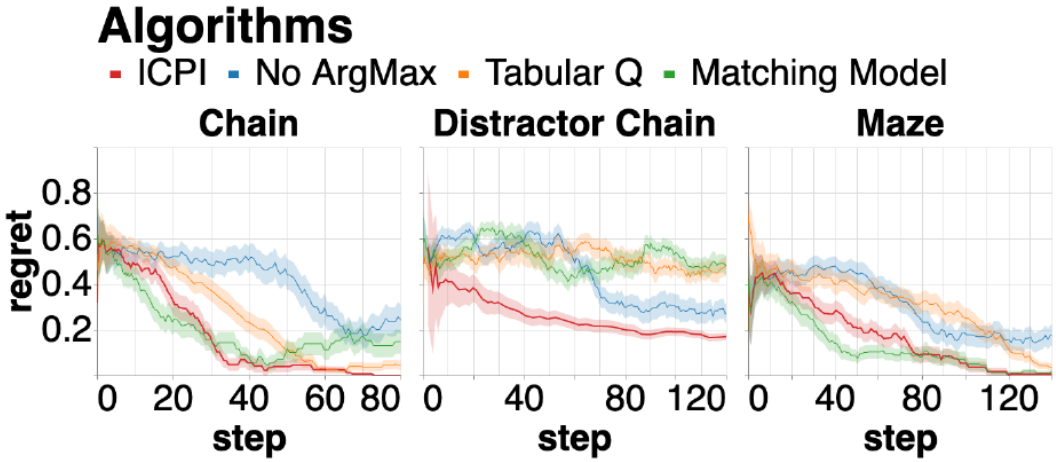
Why TD-learning?

RL with LLM[2]

Initialization: Q value table. For $t = 1, \cdots, T$:

- **Learner:** Take greedy action a_t according to Q table
- **Environment:** Observe the reward $R_t(s_t, a_t)$ and the next state s_{t+1}.
- **Update:** $Q(s_t, a_t) = \text{LLM}(\{s_t, a_t, R_t\})$.

Just use LLM to do policy evaluation and improvement?



[2]E. Brooks, et.al. Large Language Models Can Implement Policy Iteration NeurIPS, 2023.

Policy Gradient in RL: An Optimization View

Online gradient descent:

$$x_{t+1} = x_t - \eta \nabla f_t(x_t).$$

Policy gradient (maximize reward):

$$\pi_{t+1} = \pi_t + \eta \nabla V^{\pi_t}(\rho),$$

where ρ is the initial state distribution.

Q: How to compute the policy gradient? or how to estimate the policy gradient from the data?

A: Policy gradient Theorem by Rich Sutton, et al.[3]

[3] Rich Sutton, et al. "Policy Gradient Methods for Reinforcement Learning with Function Approximation", NIPS 2000.

Policy Gradient Theorem in RL

Theorem 3 (Policy Gradient Theorem)

Let π_θ be a policy parameterized by θ for a γ-discounted Markov decision process, we have the policy gradient to be

$$\nabla_\theta V^{\pi_\theta}(\rho) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_\rho^{\pi_\theta}, a \sim \pi_\theta(\cdot|s)} \left[Q^{\pi_\theta}(s,a) \nabla_\theta \log \pi_\theta(a|s) \right].$$

Policy Gradient Theorem is the foundation for policy gradient type of algorithms (e.g., Actor Critic, Natural Policy Gradient, TRPO, PPO).

Make PG practical: from TRPO to PPO

Trust Region Policy Optimization (TRPO) improves vanilla policy gradient by imposing KL constraint:

$$\max_{\theta} \mathbb{E}_{(s,a)\sim\pi_{\theta_{\text{old}}}} \left[\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}(s,a) \right]$$
$$\text{s.t. } \mathbb{E}_{s\sim\pi_{\theta_{\text{old}}}} \left[\text{KL}(\pi_{\theta_{\text{old}}}(\cdot|s) \| \pi_\theta(\cdot|s)) \right] \leq \delta$$

Proximal Policy Optimization (PPO) simplifies TRPO with a clipped surrogate objective:

$$\max_{\theta} \mathbb{E} \left[\min \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} A^{\pi_{\theta_{\text{old}}}}, \text{ clip} \left(\frac{\pi_\theta(a|s)}{\pi_{\theta_{\text{old}}}(a|s)}, 1-\epsilon, 1+\epsilon \right) A^{\pi_{\theta_{\text{old}}}} \right) \right]$$

- An approximation of TRPO.
- KL divergence is approximated by simply clipping the policy gradient.

Policy Gradient in RL: An Optimization View

Online gradient descent:

$$x_{t+1} = x_t - \eta \nabla f_t(x_t).$$

Policy gradient (maximize reward):

$$\pi_{t+1} = \pi_t + \eta \nabla V^{\pi_t}(\rho).$$

Online mirror-descent gradient:

$$x_{t+1} = \arg\min_{\mathcal{K}} \langle x, \ell_t \rangle + \frac{1}{\eta} B_\psi(x; x_t).$$

Policy mirror-descent gradient:

$$\pi_{t+1} = \arg\min_{\pi \in \mathcal{K}} \langle \pi, -\nabla V^{\pi_t}(\rho) \rangle + \frac{1}{\eta(1-\gamma)} B_\psi(\pi; \pi_t).$$

Policy Gradient in RL: An Optimization View

Policy mirror-descent gradient:

$$\pi_{t+1} = \arg\min_{\pi \in \mathcal{K}} \langle \pi, -\nabla V^{\pi_t}(\rho) \rangle + \frac{1}{\eta(1-\gamma)} B_\psi(\pi; \pi_t).$$

We need policy gradient theoremto compute $\nabla V^{\pi_t}(\rho)$.

Policy mirror-descent gradient for any state s:

$$\pi_{t+1}(s) = \arg\min_{\pi(s) \in K(s)} \langle \pi(s), -Q^{\pi_t}(s) \rangle + \frac{1}{\eta} B_\psi(\pi(s); \pi_t(s)).$$

It is very similar with what we have discussed e.g., online gradient descent, Hedge, EXP3.

Policy Gradient in RL: An Optimization View

Let ψ be $\|\cdot\|^2$. Policy mirror-descent gradient returns

$$\pi_{t+1}(s,a) = \pi_t(s,a) + \eta Q^{\pi_t}(s,a).$$

Let ψ be negative entropy and $B(\cdot,\cdot)$ becomes KL divengence. Policy mirror-descent gradient returns "EXP3"

$$\pi_{t+1}(s,a) = \pi_t(s,a)\frac{e^{\eta Q^{\pi_t}(s,a)}}{\sum_{a'} e^{\eta Q^{\pi_t}(s,a')}},$$

which has a famous name "Natural Policy Gradient"[4].

[4] Sham Kakade. "A Natural Policy Gradient". NIPS, 2021

Theoretical Results of Natural Policy Gradient

Theorem 4

Under natural policy gradient or policy mirrored descent and a constant learning rate η, we have

$$\sum_{t=1}^{T} V^{\pi^*}(s) - V^{\pi_t}(s) = O(1).$$

- The convergence rate is $O(1/T)$.
- The convergence rate is dimension-free.
- Does it contradict the previous $O(\sqrt{T})$?
- Alternative way to write natural policy gradient is

$$\pi_{t+1} = \pi_t - \eta F^{\dagger} \nabla V^{\pi_t}(\rho),$$

where F is Fisher matrix $F = \mathbb{E}[\nabla \log \pi \cdot (\nabla \log \pi)^T]$.